

A TWO-DAY HANDS-ON WORKSHOP

AI + DOCUMENT INTELLIGENCE

Taming Unstructured Data — From Prompt to Production

Building, Debugging, and Deploying a Real Document-Intelligence System with LLMs, OCR, and Docker

8 HOURS · 2 DAYS · 4 HOURS/DAY

A real, working, production codebase —
with real bugs, real fixes, and a real Docker deployment at the end.

[Workshop Dates_To_Be_Decided_By_Institute] · [Venue / Lab]

Contents

Contents	2
Problem Statement	3
Why AI in Production Is Hard	3
Agenda Of This Workshop.....	6
What Students Will Learn	6
By the end of Day 1	6
By the end of Day 2	7
Workshop Format & Detailed Agenda	8
Day 1 — Teaching a Machine to Read.....	8
Day 2 — Making It Reliable, and Shipping It.....	8
Lab Requirements	10
Per Student — Hardware & Software	10
Provided by the Organizers.....	10
Who Should Attend.....	11
Tools & Technologies Covered	11
What Students Walk Away With.....	12

Problem Statement

Every year, government departments publish tens of thousands of tenders — dense, 30-to-100-page PDFs packed with eligibility criteria, terms and conditions, and pricing schedules. Today, extracting the handful of structured facts buried in each one — the tender number, deadlines, approved brands, quantities and rates for every item — is almost entirely manual, done by hand, tender after tender. A missed clause can disqualify a bid; a misread quantity can cost lakhs / crores. It does not scale.

The data extraction problem is prevalent in various domains like Academia where Researchers and Professors have to extract certain key information from many research papers. The extract needs to be accurate with zero chance of error.

Turning “read this PDF and extract these fields” into a system that works reliably, cheaply, and repeatably on real, messy, scanned documents is one of the hardest and most instructive problems in applied AI engineering today — and by the end of Day 2, students will have built, debugged, and deployed exactly that system, themselves.

Why AI in Production Is Hard

Ask any team that has actually shipped an AI system — not demoed one — and you will hear the same story: the model was never the hard part. Here is what students will run into, first-hand, because it is what actually happened while building the system this workshop is based on.

- **Unstructured, inconsistent source data.** Real documents are not clean text files. They are scanned PDFs, mixed digital and scanned pages, tables that do not line up to a grid, and OCR artifacts like stray “:selected:” markers from misread checkboxes. Same information — like “Pricing schedule” — could be under many different names: “Schedule of Quantities,” “Schedule of Work,” “Bill of Quantities”, “BOQ”, etc.

- **Choosing — and trusting — the right Document Intelligence model.** OCR is not a solved problem. There is a real engineering choice between generic text OCR and layout-aware models (like Azure AI Document Intelligence's prebuilt-layout model) that understand tables as rows and columns instead of scrambled text. Even the best OCR occasionally drops a digit or merges a cell — errors that quietly ripple downstream if nobody is watching for them.
- **Token limits and the economics of context.** You cannot always hand an entire 100-page document to an LLM in one call. Cost, latency, and context-window limits force real engineering decisions: how do you split a document into batches without slicing a table in half? How do you merge results back together across those batch boundaries without losing or duplicating data?
- **LLMs are non-deterministic.** The same prompt, on the same input, run twice, can produce two different structured outputs — a fact most AI demos never show, because they only ever run once. This breaks the standard “build it, test it, ship it” mental model most students already have: you cannot unit-test a prompt the way you unit-test a function.
- **Extraction is classification wearing a disguise.** Every “extract X from this document” task hides a classification problem: does this table belong to the pricing schedule, or is it an unrelated rate table that happens to sit on a similar-looking page? Getting the data right is only half the job — getting it into the right bucket is where most real bugs actually live.
- **Cost is an engineering constraint, not an afterthought.** OCR is billed per page. LLM calls are billed per token. A system processing thousands of documents a month needs monthly budgets, usage tracking, and the discipline not to re-process the same document twice by accident.
- **Trust, auditability, and “why did it do that?”** When an AI system gets something wrong — and it will — you need to be able to answer why. That means keeping an audit trail of exactly what was extracted, from which page, by which model call, not just handing back a final answer and hoping it is right.
- **The gap between a notebook and a production system.** A working prototype in a Jupyter notebook and a production pipeline that survives

concurrent uploads, partial failures, retries, and real deployment are two very different engineering problems. Most AI coursework stops at the first one. This workshop starts where that coursework ends.

- **API and Storing Secret Keys.** Students will create API keys for different LLM and will store them safely. The keys and environment variables for development and production are stored differently in each and provide a security barrier between end users and developers so that secret keys do not get exposed.

Agenda Of This Workshop

Students will design a cloud application that

1. asks user for login,
 2. user uploads a PDF documents,
 3. runs a pipeline (takes 3 – 5 minutes because of complexity of task),
 4. user gets downloadable extract in form of MS Excel multi-sheet and
 5. user also receives an automated email containing downloadable URL link valid for 24 hours.
- **A real, evolving codebase, not toy examples.** Every exercise runs against the same production-grade pipeline used to process real government tenders — not a simplified stand-in built for the classroom.
 - **Real bugs, on purpose.** Students will reproduce and fix actual issues this system hit in production: an LLM that quietly duplicated table rows, one that misfiled a signature block as contract data, one that fabricated a list from a blank certificate template. Each is a genuine lesson in how AI systems fail — and how to defend against it.
 - **End-to-end, not just the model.** Ingestion, OCR, structured extraction, validation, Excel export, job tracking, and a live Docker deployment — the whole system, the way it actually ships.
 - **Engineering discipline applied to AI.** Testing strategy, cost governance, observability, and deterministic guardrails around a non-deterministic core — the parts of AI engineering that separate a prototype from a product.

What Students Will Learn

By the end of Day 1

- How OCR and Document Intelligence models turn a scanned PDF into structured, machine-readable text and tables.

- How to design prompts and schemas that make a large language model return reliable, structured data — not free-form prose.
- Why long documents must be chunked before reaching an LLM, and how to merge results back together across chunk boundaries without losing information.
- The anatomy of a real extraction pipeline: ingest → OCR → structured extraction → validation → export, as a sequence of small, testable stages.

By the end of Day 2

- Why AI systems fail differently from ordinary software — non-determinism, hallucination, and misclassification — and how to reason about failures you cannot fully predict.
- How to build deterministic guardrails and validators around a non-deterministic AI core, so the system fails loudly instead of silently.
- What can and cannot be unit-tested in an AI pipeline, and how to design regression tests around real, previously-seen failures.
- How to track cost and usage (pages OCR'd, tokens spent) and enforce a budget in a production system.
- How to containerize a multi-service AI application with Docker and docker-compose, and deploy it end to end.

Workshop Format & Detailed Agenda

Two days, four hours each, split roughly evenly between concepts and hands-on labs — every lab builds directly on the last.

Day 1 — Teaching a Machine to Read

Time	Session	Format
0:00 – 0:20	Welcome & the Problem Statement Why manual tender extraction breaks down at scale — and why “just ask an LLM” is not the whole answer.	Talk
0:20 – 1:00	Anatomy of a Document-Intelligence Pipeline OCR, structured extraction, schemas, and how the pieces fit together end to end.	Talk + Demo
1:00 – 1:15	Break	—
1:15 – 2:15	Lab 1 — OCR a Real Tender Run Document Intelligence on a real scanned tender PDF and inspect the raw structured output.	Hands-On Lab
2:15 – 3:15	Lab 2 — Your First Structured Extraction Write a prompt and a schema; extract real fields from a real document and see what comes back.	Hands-On Lab
3:15 – 3:45	Token Limits, Chunking & Merging Why you can’t send 60 pages in one call, and how results get stitched back together.	Talk + Discussion
3:45 – 4:00	Day 1 Wrap-Up What we built, what already looks fragile, and what Day 2 is going to break on purpose.	Discussion

Day 2 — Making It Reliable, and Shipping It

Time	Session	Format
0:00 – 0:20	Recap: What Went Wrong Overnight A real extraction bug from this system’s history, run live in front of the room.	Live Demo
0:20 – 1:00	Why AI Systems Fail Differently Non-determinism, hallucination, and misclassification — with real, reproducible examples.	Talk + Demo
1:00 – 1:15	Break	—

Time	Session	Format
1:15 – 2:15	Lab 3 — Fix a Real Misclassification Bug Diagnose why the model misfiled content, then build a deterministic guardrail that catches it for good.	Hands-On Lab
2:15 – 2:45	Testing AI Pipelines What can be unit-tested, what can't, and how regression tests are written around real past failures.	Talk + Lab
2:45 – 3:00	Cost Governance Tracking pages OCR'd and tokens spent; enforcing a monthly budget in a live system.	Talk
3:00 – 3:45	Lab 4 — Containerize & Deploy Package the system with Docker and docker-compose and bring it up end to end, locally.	Hands-On Lab
3:45 – 4:00	Live Demo, Q&A & Closing Upload a real tender and watch it flow through the entire system to a validated Excel report.	Demo + Q&A

Lab Requirements

Per Student — Hardware & Software

Requirement	Details
Laptop	Windows, macOS, or Linux — minimum 8 GB RAM (16 GB recommended), 10 GB free disk space, able to run Docker Desktop.
Internet Access	Reliable Wi-Fi — required for cloud API calls (Azure OpenAI, Document Intelligence) and pulling Docker images.
Docker Desktop	Installed and running before Day 1 (Windows users: with WSL 2 backend enabled).
Python 3.11+	Installed and available on PATH.
Git	Installed, with a GitHub account to clone the workshop repository.
Code Editor	Visual Studio Code recommended, with the Python extension installed.
Azure Access	An Azure subscription with access to Azure OpenAI Service and Azure AI Document Intelligence — or sandbox credentials provided by the organizers (recommended, to avoid approval-wait delays for Azure OpenAI access).

Provided by the Organizers

Provided	Details
Starter Repository	GitHub repository with the full pipeline, sample tender PDFs, and lab exercises for both days.
Sandbox API Keys	Shared, rate-limited Azure OpenAI and Document Intelligence credentials for the duration of the workshop, so students without their own Azure access are not blocked.
Pre-Pulled Docker Images	Base images cached on the lab network in advance, to keep Lab fast even on limited bandwidth.
Sample Tender PDFs	A small set of real, anonymized government tender documents of varying complexity for the hands-on labs.

Who Should Attend

- **Computer Science, IT, and Data Science students** with basic working knowledge of Python (functions, loops, calling a library). No prior machine learning or AI coursework required.
- **Students** exploring internships or projects in applied AI, backend engineering, or cloud systems.
- **Faculty** looking to bring applied AI engineering — not just ML theory — into project courses, capstones, or electives.

Tools & Technologies Covered

- **Azure OpenAI Service** — large language model calls for structured field extraction.
- **Azure AI Document Intelligence** — layout-aware OCR: text, tables, and key-value pairs from scanned PDFs.
- **Python, FastAPI & Pydantic** — the REST API and the structured-output schemas that keep the LLM honest.
- **Streamlit** — a lightweight browser front end talking to the API over plain HTTP.
- **SQLite** — lightweight, file-based job tracking and status history.
- **Docker & docker-compose** — containerizing and running the full multi-service system end to end.
- **pytest** — regression testing for both deterministic code and AI-pipeline guardrails.
- **Login** — A login interface that is front end of application

What Students Walk Away With

1. **A working, end-to-end AI system on their own machine.** Cloned, built, and run by them — not watched on a projector.
2. **Hands-on experience with the exact stack** used in real production AI systems today: a cloud LLM, a cloud OCR service, a REST API, and a container deployment.
3. **A portfolio-ready project** on their own GitHub, with a commit history they can talk through in an interview.

Q&A: At the End

1. Find the initial setup cost + running-cost on a 100-page PDF document?
2. What other models are out there in case the current models get deprecated?
4. What are the strength and weakness of the current architecture and steps to enhance this architecture?
3. Q&A: Other questions that arise during the delivery.